

Atividade Pratica - DevOps

Plataforma fullstack para gestao de clientes, favoritos e compras, com frontend web, API REST, integracao com Fake Store API, autenticacao, painel administrativo, persistencia em Firebase Realtime Database, containerizacao e CI/CD.

Visao geral

O projeto foi construido para cobrir um fluxo completo de produto:

- autenticacao e sessao de usuario
- gestao de clientes
- catalogo de produtos com filtros
- favoritos/carrinho
- compra com PIX e QR Code
- administracao de usuarios e produtos
- persistencia em banco realtime
- deploy e pipeline automatizada

Stack tecnica

- Frontend: React, TypeScript, SASS
- Backend: Node.js, TypeScript, Hono
- Banco principal: Firebase Realtime Database
- Auth: Firebase Auth (com fallback local)
- API externa: Fake Store API
- Containerizacao: Docker e Docker Compose
- CI/CD: GitHub Actions
- Deploy alvo: Cloudflare Workers (backend), Cloudflare Pages/Netlify (frontend)

Criadores do Projeto

- 37023164 FÁBIO LEON BARBOSA TAVARES (documentacao e auxiliar)
- 37023114 JOAB RANIEL RODRIGUES (Docker)
- 37022679 GUILHERME LOPES S. DA CRUZ (Back End e revisao)
- 37023010 ISAIAS LEVY TAVARES DA SILVA (FronD End)

Funcionalidades completas

1. Identidade visual e carregamento inicial

- Logo aplicada nas telas principais (login e app)
- Nome do produto exibido na interface
- Favicon configurado
- Metadados de compartilhamento (Open Graph e Twitter Card)

2. Autenticacao e contas

- Login de usuario
- Cadastro de nova conta
- Integracao com Firebase Auth
- Fallback local quando auth remoto nao esta disponivel
- Controle de sessao com persistencia local
- Logout com tentativa de encerramento de sessao remota

3. Termo de responsabilidade

- Checkbox obrigatorio no login
- Modal com leitura do termo
- Bloqueio de acesso enquanto nao houver aceite

4. Perfil e navegacao da aplicacao

- Sidebar com dados do usuario autenticado
- Contador de favoritos/carrinho
- Navegacao entre secoes:
 - catalogo
 - carrinho
 - pagamento
 - painel admin

5. Catalogo de produtos

- Consumo da Fake Store API
- Renderizacao de cards de produtos
- Busca por texto
- Filtro por categoria
- Traducao de categorias para pt-BR
- Atualizacao visual em tempo real conforme filtros

6. Favoritos e carrinho

- Favoritar produto a partir do catalogo
- Remover dos favoritos
- Evita duplicidade no carrinho
- Pagina dedicada de carrinho
- Persistencia local do carrinho
- Sincronizacao de favoritos por cliente via backend

7. Pagamento via PIX

- Geracao de chave/transacao dinamica
- Geracao de payload BR Code (EMV)
- Calculo de CRC16 para payload
- Exibicao de QR Code para pagamento
- Copia da chave/payload
- Registro de compra no backend apos iniciar pagamento

8. Gestao de clientes (CRUD)

- Criar cliente
- Listar clientes
- Editar cliente
- Excluir cliente
- Regra de e-mail unico
- Mensagens de erro/sucesso por operacao

9. Painel administrativo

- Acesso controlado por usuario admin
- Abertura via botao dedicado (engrenagem)

- Gestao de usuarios:
 - editar nome, email e senha local
 - excluir usuario
- Gestao de produtos:
 - editar titulo, preco, categoria, imagem, descricao
 - editar rating
 - salvar override de produto

10. Persistencia de dados

- Clientes salvos no Firebase Realtime Database
- Favoritos salvos no Firebase Realtime Database
- Compras salvas no Firebase Realtime Database
- Leitura sincronizada do backend para o frontend
- Fallback para armazenamento em memoria (backend) e localStorage (frontend) quando necessario

Arquitetura de persistencia

Backend

- Repositorio por interface (`ClientRepository`)
- Implementacoes:
 - `FirebaseRepository` (producao)
 - `InMemoryRepository` (fallback/dev)
- Se `FIREBASE_DATABASE_URL` e `FIREBASE_SERVICE_ACCOUNT_JSON` estiverem definidos, usa Firebase automaticamente.
- Migracao de legado aplicada:
 - clientes consolidados no no `clients`
 - no legado `atletas` removido com migracao segura para `clients`
 - operacao atual do backend apenas no no `clients`

Frontend

- Sessao, carrinho e configuracoes locais em localStorage
- Cadastro e leitura sincronizada com endpoints do backend
- Uso de Firebase Auth para autenticacao quando disponivel
- Ajustes de robustez aplicados:
 - sincronizacao automatica de favoritos/compras com backend
 - auto-refresh de clientes/favoritos na UI
 - mitigacao de cache de assets com versionamento do `app.js`

Endpoints da API

Health

- GET `/health`

Clientes

- POST `/api/clients`
- GET `/api/clients`
- PUT `/api/clients/:id`
- DELETE `/api/clients/:id`

Favoritos

- POST /api/clients/:id/favorites
- GET /api/clients/:id/favorites
- DELETE /api/clients/:id/favorites/:productId

Compras

- POST /api/clients/:id/purchases
- GET /api/clients/:id/purchases

Estrutura de pastas

- backend/ : API REST, repositórios, configuração de runtime/deploy
- frontend/ : aplicação web, estilos, scripts e assets
- .github/workflows/ : CI e automações de deploy
- docker-compose.yml : orquestração local dos serviços

Executar localmente

Opção 1: Docker Compose

1. Na raiz do projeto:

```
docker compose up --build
```

2. Endereços:

- Frontend: <http://localhost:8080>
- Backend: <http://localhost:3000>
- Healthcheck: <http://localhost:3000/health>

Opção 2: Sem Docker

Backend:

1. Entre em backend/
2. Instale dependências:

```
npm install
```

3. Rode em desenvolvimento:

```
npm run dev
```

4. Build e start:

```
npm run build
```

```
npm start
```

Frontend:

- Entre em frontend/ .
- Instale dependências:

```
npm install
```

- Rode em desenvolvimento:

```
npm run dev
```

- Para gerar a versao de producao:

```
npm run build
```

- O Vite sobe em uma porta local exibida no terminal; ajuste a URL da API no app se necessario.
- Acesse preferencialmente:

```
http://localhost:5173
```

(evite misturar `localhost` e `127.0.0.1` na mesma sessao para reduzir efeitos de cache/origem)

Configuracao Firebase

Arquivo de exemplo: `backend/.env.example`

Arquivo real: `backend/.env`

Variaveis obrigatorias para Realtime Database:

- `FIREBASE_DATABASE_URL`
- `FIREBASE_SERVICE_ACCOUNT_JSON`

Variaveis adicionais:

- `PORT`
- `CORS_ORIGIN`

Configuracao recomendada de CORS para desenvolvimento:

- `CORS_ORIGIN=http://localhost:8080`

Observacao: o backend aceita `http://localhost:8080` e `http://127.0.0.1:8080` na resolucao de CORS.

Observacao: o backend usa `dotenv/config`, portanto carrega o `.env` local automaticamente ao iniciar.

Atualizacoes recentes

- Correcao de sincronizacao realtime entre frontend e backend para favoritos e compras.
- Correcao de fluxo de cadastro para reduzir falso positivo de e-mail duplicado em ambiente local.
- Bust de cache no frontend para garantir carregamento da versao mais nova do `app.js`.
- Migracao concluida do no legado `atletas` para `clients` no Firebase Realtime Database.
- Ajuste de CORS para cenarios de desenvolvimento local.
- Cadastro ajustado para persistir conta no Realtime Database mesmo sem URL da API configurada.
- Fluxo de login com fallback local quando autenticacao remota estiver indisponivel.
- Carrinho isolado por usuario para evitar compartilhamento indevido de itens entre contas.

CI/CD

Pipeline de CI

Arquivo: `.github/workflows/ci.yml`

Etapas principais:

- instalacao de dependencias
- checagem TypeScript
- build backend
- build de imagens Docker

Pipeline de deploy

Arquivo: `.github/workflows/deploy-cloudflare.yml`

Possibilidades:

- deploy de backend no Cloudflare Workers
- deploy de frontend no Cloudflare Pages

Publicacao frontend (Netlify)

Para o frontend no Netlify, o projeto ja possui:

- `<title>` definido
- favicon com a logo
- metadados de preview (og e twitter)

Importante:

- publicar o conteudo de `frontend/`
- manter o arquivo da logo em `frontend/public/img/`
- configurar a variavel de ambiente `VITE_API_BASE_URL` com a URL publica do backend (exemplo: `https://seu-backend.exemplo.com`)
- para operacoes completas (listagem/edicao/remocao de clientes e favoritos), configurar URL base da API no app apos publicar
- para cadastro inicial de conta, o frontend ja possui fallback direto ao Realtime Database

Validacao funcional recomendada

Checklist rapido:

1. Cadastro de usuario novo
2. Login com aceite de termo
3. Criacao de cliente
4. Favoritar produto
5. Abrir carrinho e comprar via PIX
6. Confirmar registro em:
 - `/clients`
 - `/favorites`
 - `/purchases` no Firebase Realtime Database

Entregaveis

- Repositorio GitHub: <https://github.com/Fabio684/Atividade-Pr-tica---DevOps.git>
- URL do frontend publicado: <https://joyful-madeleine-cbb9bc.netlify.app>
- URL da API publicada: <https://favorites-api.devops-api.workers.dev>

Status de producao (25/05/2026)

- Frontend publicado em Netlify com deploy ativo:
 - <https://joyful-madeleine-cbb9bc.netlify.app>
- Backend publicado em Cloudflare Workers com healthcheck OK:
 - GET <https://favorites-api.devops-api.workers.dev/health> -> {"status":"ok"}
- Variavel de ambiente no Netlify configurada:
 - `VITE_API_BASE_URL=https://favorites-api.devops-api.workers.dev`
- Publicacao do frontend concluida via CLI com upload direto do dist:
 - `netlify deploy --prod --no-build --dir=C:\temp\distdeploy`

Firebase x Netlify (verificacao)

- Firebase Authentication:
 - provedor `E-mail/senha` esta ativado
- Firebase Auth > Configuracoes > Dominius autorizados:
 - dominios presentes: `localhost` , `devops-57d46.firebaseio.com` , `devops-57d46.web.app` , `joyful-madeleine-cbb9bc.netlify.app`
 - status: dominio do deploy Netlify adicionado com sucesso como dominio `Custom`
- Impacto:
 - login/cadastro no frontend publicado fica alinhado com a configuracao de auth para redirecionamentos suportados
 - reduz risco de bloqueio de fluxo de autenticacao em producao

Validacao do tutorial (Notion)

Comparativo do tutorial com o que foi executado:

- `git status --short --branch` : executado
- `git status --ignored --short` : executado e `.env` confirmado como ignorado (`backend/.env`)
- `git fetch origin` : executado
- `npm install` e `npm run build` no frontend: executados com sucesso

Conclusao: o procedimento do tutorial foi implementado e validado no repositório atual.

Troubleshooting rapido

- Mensagem "Ja existe uma conta com esse e-mail":
 - confirme se o e-mail ja existe em `GET /api/clients`
 - reinicie o frontend sem cache (`-c-1`)
 - recarregue a pagina com hard refresh
- Mensagem "Nao foi possivel criar conta agora":
 - confira conectividade com Firebase
 - valide configuracao/estado do Firebase Auth e regras do Realtime Database
 - tente novamente apos hard refresh para garantir versao mais recente do frontend
- Erro de CORS em ambiente local:
 - confira `CORS_ORIGIN` no backend
 - use a mesma origem no navegador e no valor de CORS
- Porta ocupada (EADDRINUSE):

- finalize processo na porta 3000/8080 antes de subir novamente os servicos

Observacoes finais

- O projeto suporta modo online (Firebase) e modo fallback (local/memoria).
- O cadastro foi reforçado para priorizar persistencia no Realtime Database no frontend publicado.
- O backend foi estruturado para facilitar evolucao para novos repositórios de dados.
- O frontend foi organizado para manter UX completa em desktop e mobile.